



Facultad de Ingeniería
Maestría en Ciencia de Datos – 2024/2025

Fundamentos del Aprendizaje Automático
Trabajo Práctico Final

Integrantes:

- Cancelas, Martín.
- Filipuzzi, Juan Manuel.
- Gallardo, Ezequiel.
- Nicolau, Jorge.

Contenido

Planteo del Problema	2
<i>Resolución del Problema Dataset</i>	2
<i>Resolución del Problema Código</i>	2
Preprocesamiento	3
Análisis exploratorio de datos <i>Auto EDA</i>	3
<i>YData Profiling</i>	4
<i>SweetViz</i>	6
<i>AutoViz</i>	10
<i>D-Tale y Dataprep</i>	13
Ingeniería de Características <i>Auto Feature Engineering</i>	13
<i>TsFresh</i>	13
<i>FeatureTools</i>	14
<i>AutoFeat</i>	14
<i>AutoClean</i>	16
Selección de Modelo <i>AutoML</i>	18
H2O	18
<i>PyCaret</i>	19
<i>AutoKeras</i>	19
Optimización de Hiperparámetros <i>AutoHPO</i>	20
H2O <i>Grid Search</i>	21
<i>Optuna</i> Optimización Bayesiana	21
<i>Optuna</i> <i>Random Search Hyperband</i>	22
Evaluación Final de los modelos	23
Automatización del proceso <i>MLOps</i>	24
Solución <i>MLOps</i>	25
<i>MLFlow</i>	26
<i>GoCD</i>	29
Problemas de versionado de <i>Python</i> Planteo de solución	30
Conclusiones	31
Referencias	33
Anexo 1: CANVAS Business Model	35

Planteo del Problema

Una empresa de medicina prepaga cuenta con tres sanatorios propios y una compañía de seguros de salud que articula la atención médica de sus socios. El circuito de atención comienza con los llamados que realizan los afiliados solicitando asistencia a domicilio, ya sea en forma de emergencias, urgencias o visitas médicas programadas. A partir de esta demanda inicial se decide el envío de ambulancias cuando corresponde, y en los casos más complejos se gestiona la internación de los pacientes. El ideal estratégico de la compañía es que el cien por ciento de las internaciones se realice en los sanatorios del propio grupo, evitando derivaciones a clínicas externas que implican mayores costos, menor control operativo y pérdida de ingresos.

El problema surge porque la cantidad diaria de internaciones no programadas es altamente variable, lo que dificulta la planificación de la disponibilidad de camas y la organización de cirugías electivas. Esta incertidumbre genera tensiones: en los días con elevada demanda puede ser necesario derivar pacientes fuera de la red, mientras que en los días con baja demanda quedan recursos ociosos sin aprovechar. En este contexto se plantea como solución el desarrollo de un sistema predictivo que, a partir de la serie temporal de internaciones históricas, el volumen de servicios médicos prestados por la prepaga y la evolución del número de socios permita anticipar la cantidad futura de internaciones totales. Con un pronóstico confiable, la empresa estaría en condiciones de programar con mayor eficiencia la utilización de camas, ajustar la agenda de cirugías y definir estrategias operativas para balancear la demanda entre sus distintas unidades.

La construcción de un modelo predictivo de estas características tendría beneficios múltiples. No sólo permitiría reducir la proporción de derivaciones a instituciones externas, sino también optimizar la utilización de quirófanos, mejorar la eficiencia de los recursos humanos y materiales y asegurar un mejor servicio a los afiliados, que accederían con mayor probabilidad a una cama dentro de la red propia.

Resolución del Problema | Dataset

El conjunto de los datos de negocio utilizado se encuentra alojado en https://raw.githubusercontent.com/georgsmeinung/mlops-pipeline/refs/heads/main/data/df_automl-serie_diaria.csv.

Resolución del Problema | Código

El código con la aplicación de todo lo descrito en el presente trabajo se encuentra desarrollado en https://colab.research.google.com/drive/1_zktH-prnDTbEyi0vcOvvcJ27ov3sBsz.

Preprocesamiento

Para la captura del conjunto de datos se utilizó la librería *dask*, la cual, tal como detalla su propia documentación (2025), es una biblioteca de *Python* para computación paralela y distribuida. El beneficio de esta, a comparación de la utilización tradicional de *pandas*, es la reducción del uso de memoria para grandes conjuntos de datos gracias a, precisamente, su utilización en paralelo y de forma distribuida. A pesar de ello, en este caso particular, el conjunto de datos ha ocupado poca memoria, alcanzando, aproximadamente, los 0,27MB.

Durante esta etapa se observaron y realizaron dos ajustes: en primer lugar, algunas variables del conjunto se importaron con un formato de datos erróneo, en este caso como *string* siendo realmente numéricas, por lo que todas fueron convertidas a *int64*; en segunda instancia, con el objetivo de optimizar el uso de memoria aún más, se verificó que todas las variables numéricas, que inicialmente tenían formato de datos *int64*, pudieran convertirse a un formato numérico que ocupe un menor espacio, lo que derivó en la conversión de 27 variables enteras en 64 bits a 2 en 32 bits, 11 en 16 bits y 14 en 8 bits, logrando una reducción del uso de espacio en memoria del 77,93%.

Análisis exploratorio de datos | *Auto EDA*

En el contexto de *AutoML*, el término *AutoEDA* (*Exploratory Data Analysis* automatizado) se refiere a la aplicación de técnicas y algoritmos que permiten realizar de manera automática el análisis exploratorio de los datos antes del entrenamiento de modelos. Su propósito es generar una caracterización sistemática del conjunto de datos mediante la identificación de tipos de variables, distribución de valores, presencia de valores atípicos o faltantes, correlaciones relevantes y posibles problemas de calidad. Este proceso, que en un flujo tradicional requeriría un trabajo manual intensivo, se automatiza para proveer reportes interpretables, detectar sesgos iniciales y orientar decisiones posteriores del motor *AutoML*, tales como la selección de transformaciones, la imputación de datos o la generación de *features*. De este modo, el *autoEDA* constituye un componente crítico para garantizar que los modelos producidos por *AutoML* se construyan sobre datos consistentes y comprendidos en su estructura estadística básica.

Distintas librerías generan, con muy poca codificación, una variedad importante de figuras, principalmente dependiendo del tipo de dato de cada variable del conjunto de datos. Se incluyen representación rápida de la distribución de cada una, estadísticas descriptivas, matrices de correlación y otras visualizaciones que permiten un veloz repaso por sobre todo el conjunto para observar potenciales asociaciones. Durante el presente, se exploraron las alternativas de *ydata profiling*, *sweetviz*, *autoviz*, *dtale* y *dataprep*.

YData Profiling

La documentación de esta herramienta (2025) detalla que automatiza y estandariza la generación de informes detallados, con estadísticas y visualizaciones, y su importancia radica en cómo optimiza el proceso de comprensión y preparación de datos para su análisis con, solamente, una línea de código para la generación de un reporte en formato HTML. Otra opción para incluir el análisis en un pipeline de datos es generar el reporte en formato JSON. Entre las características a destacar de este paquete se encuentran la descripción del conjunto de datos, de las variables, la comparación de múltiples versiones de un mismo conjunto de datos y complementar el informe con detalles del conjunto de datos y de cada variable.

El objetivo del reporte de YData Profiling es resumir las características estructurales, la calidad de los datos y las principales conclusiones descriptivas de las variables, especialmente aquellas destinadas al modelado de series temporales. El *dataset* analizado exhibe una excelente calidad y una estructura robusta para el análisis estadístico. El conjunto consta de 1186 observaciones (filas) y 28 variables (columnas), abarcando un periodo diario continuo. El hallazgo más significativo en términos de integridad es la ausencia total de valores faltantes (*p_cells_missing*: 0.0), lo que elimina la necesidad de procedimientos de imputación y garantiza la completitud del análisis. En cuanto a su composición, el *dataset* está dominado por 27 variables de tipo numérico, acompañadas por una única variable de fecha y tiempo (*DateTime*), lo cual es un indicador claro de que está diseñado para el análisis de series temporales.

La variable día actúa como la columna clave temporal del *dataset*. El reporte confirma que esta variable es única para cada una de las 1186 observaciones (*is_unique*: true), estableciendo que cada registro representa un evento o medición por día. Esto valida el *dataset* como una serie de tiempo diaria bien estructurada. Al ser un identificador temporal completo y continuo, la variable día está lista para indexar modelos de predicción, permitiendo la descomposición de tendencias y estacionalidades en las métricas numéricas asociadas.

Las 27 variables numéricas contienen las métricas operacionales o de interés, como *temp_max* y diversas variables de tiempo (e.g., *prom_arribo_min_amarillo*), que probablemente miden duraciones o esperas en minutos. El perfilado revela un patrón de distribución consistente en varias de estas métricas, caracterizado por una asimetría positiva significativa (*skewness* alta), a menudo superior a 3.5, y una alta curtosis (valores superiores a 14). Esta distribución fuertemente asimétrica tiene implicaciones cruciales: indica que la gran mayoría de las mediciones son bajas y se agrupan cerca de la media o mediana, representando la operación normal o eficiente. Sin embargo, la larga "cola" de la distribución se extiende hacia valores extremadamente altos, lo que se traduce en la presencia de valores atípicos (*outliers*) que pueden llegar a ser diez o veinte veces superiores a la mediana. Estos puntos de datos anómalos (por ejemplo, tiempos máximos que

superan los 300 o 600 minutos) deben ser analizados por su impacto potencial, ya que pueden distorsionar la media y afectar negativamente el rendimiento de modelos de regresión lineal tradicionales.

El análisis detallado de la nomenclatura de las variables sugiere que el *dataset* proviene de un sistema de gestión de servicios o procesos que opera bajo un esquema de segmentación o clasificación. La variable `prom_arribo_min_amarillo` no solo cuantifica un tiempo promedio de llegada, sino que lo etiqueta con el descriptor "amarillo". Esto es un indicador robusto de que existen (o deberían existir) mediciones análogas para otros estados o categorías, como "rojo" o "verde", típicas en sistemas de semáforo, urgencias médicas, o niveles de criticidad de servicio. El descubrimiento clave aquí es que el rendimiento del proceso no es uniforme, sino que está estratificado por prioridad o fase. Esto implica que cualquier modelo predictivo debe considerar esta segmentación como un *feature* categórico (si estuviera disponible) o, al menos, que las predicciones deben interpretarse bajo el contexto específico de la categoría "amarillo", que usualmente denota una prioridad media o una situación de precaución.

Estadísticamente, el conjunto de datos está dominado por variables numéricas que presentan una asimetría positiva extrema y una alta curtosis. Esta forma de distribución no es accidental; es el patrón característico de los tiempos de espera o de duración de servicio en el mundo real. La alta asimetría (con valores de *skewness* superando 3.5 en algunos casos) significa que, si bien la mayoría de los eventos son rápidos y eficientes, agrupándose cerca de los valores mínimos, existe una "cola larga" y dispersa de eventos excepcionalmente lentos o prolongados. Estos picos, que se manifiestan como valores atípicos (máximos que superan la mediana por factores de diez o más), representan fallas, congestiones severas, o casos de servicio muy complejos. Desde una perspectiva de gestión del rendimiento, este es el descubrimiento más relevante: la oportunidad de mejora operativa radica en identificar las causas detrás de esos picos atípicos. Para la modelación, esta asimetría requiere la aplicación de transformaciones como el logaritmo para lograr una distribución más normal y evitar que estos *outliers* sesguen los parámetros del modelo.

El análisis de la configuración del reporte y la granularidad de ciertas variables ofrecen información sobre el diseño experimental. El hecho de que la herramienta de perfilado haya sido configurada con un umbral de correlación de 0.9 indica una preocupación consciente del analista por la multicolinealidad. Esto sugiere que existen sospechas fundadas sobre la redundancia de información entre algunas de las 27 variables numéricas, un problema común en la recopilación automatizada de métricas operacionales. La intención es clara: se deben identificar y simplificar las características altamente correlacionadas para construir un modelo más interpretable y numéricamente estable. En un punto relacionado, la variable `temp_max`, con solo 29 valores distintos, evidencia una discretización del dato continuo original. Esto limita la capacidad de cualquier modelo

para capturar efectos sutiles de la temperatura en el rendimiento; si la temperatura cambia marginalmente entre, digamos, 25.1°C y 25.9°C, el modelo solo percibirá un cambio cuando el valor pase de 25 a 26, ignorando la variabilidad interna que podría ser relevante.

Puede concluirse que el *dataset* está en una fase avanzada de preparación para el modelado predictivo de series de tiempo gracias a su impecable calidad de datos (cero nulos) y su estructura diaria bien definida. El principal paso de preprocesamiento requerido es la transformación de las variables numéricas fuertemente asimétricas. Para variables con una asimetría tan pronunciada, se recomienda aplicar una transformación logarítmica o de raíz cuadrada antes de la modelación. Esto ayudará a normalizar la distribución de los datos, mitigar el impacto desmedido de los *outliers* y mejorar la robustez y precisión de cualquier modelo predictivo que se construya.

SweetViz

Al igual que en el caso anterior, PyPI (2023) comenta que *SweetViz* es una biblioteca de *Python* de código abierto que genera visualizaciones atractivas y de alta densidad para iniciar el análisis exploratorio de datos con solo dos líneas de código. También destaca que el resultado es una aplicación HTML completamente autónoma. Se basa en la visualización rápida de valores objetivo y la comparación de conjuntos de datos. Su objetivo es facilitar el análisis rápido de las características del conjunto de datos. Finalmente, permite analizar valores de objetivos y su relación con otras características, comparar distintos conjuntos de datos o características intrínsecas del conjunto, asociar valores con distintos tipos de datos a través de correlación y pruebas “t”, detectar automáticamente los tipos de datos de cada característica y obtener resúmenes de estadística descriptiva de cada variable del conjunto analizado.

En el marco del presente trabajo, la utilización de esta librería generó un reporte y se adicionó una descarga automática de este para poder realizar su apertura en un navegador *web*. De lo obtenido es posible mencionar que se generaron distintas figuras que son replicables con lo observado con *YData Profiling*, por lo que, en este caso, es interesante abordar la matriz de correlaciones del conjunto de datos. Sobre el extenso reporte generado por *YData Profiling*, es posible analizar que el *dataset* contiene una variedad de variables continuas y categóricas, cuyas distribuciones se pueden observar en los gráficos proporcionados (ver archivo de reporte).

Entre las variables con sesgo positivo alto (cola larga a la derecha) encontramos *med_arribo_min_rojo* que muestra un sesgo extremadamente alto (18.8), lo que sugiere que la mayoría de los tiempos de llegada medianos para "rojo" son bajos, con algunas excepciones de valores muy altos. También *sum_cambios_color* (7.0), *cant_moli_medico_on_line* (3.8), *prom_arribo_min_rojo* (3.5), *total_cursos* (2.7), *med_arribo_min_visitas* (2.3),

prom_arribo_min_amarillo (1.8), sum_rechazos_internacion (1.7), med_arribo_min_amarillo (1.6), cant_visitas (1.3), prom_arribo_min_visitas (1.3), sum_reclamos (1.2), prom_arribo_min_moli_medico_on_line (1.0) y mediana_t_internacion_min (0.9) presentan un sesgo positivo significativo. Esto indica que la mayoría de los valores para estas métricas se concentran en el extremo inferior, con menos ocurrencias de valores elevados.

Hay una variable con sesgo negativo alto (cola larga a la izquierda): prom_t_internacion_min tiene un sesgo negativo considerable (-5.8), lo que sugiere que hay más valores altos en esta métrica, con una cola de valores más bajos.

Entre las variables con distribución aproximadamente simétrica encontramos dia_semana (0.0) y consultas_guardia (0.0) que muestran una distribución bastante uniforme o simétrica; temp_max (-0.0) y temp_min (-0.1) también son relativamente simétricas, aunque con un ligero sesgo negativo. Por último, prom_edad_soc (-0.1) es casi simétrica, con un ligero sesgo negativo.

Hay variables con sesgo positivo moderado/bajo: cant_amarillo (0.5), cant_rojo (0.5), y med_arribo_min_moli_medico_on_line (0.5) indicando una concentración en valores más bajos, pero con una distribución más extendida que las de alto sesgo. internaciones (0.4) y cant_socios (0.2) también presentan un sesgo positivo bajo.

La matriz de correlación (páginas 62-69) y los gráficos de dispersión/tendencia (páginas 69-180) revelan varias relaciones importantes:

Entre las variables con correlaciones fuertes positivas, prom_t_internacion_min y mediana_t_internacion_min presentan una correlación muy alta de 0.93. Esto sugiere que ambas métricas miden aspectos muy similares del tiempo de internación. También prom_arribo_min_rojo, prom_arribo_min_amarillo, prom_arribo_min_visitas, prom_arribo_min_moli_medico_on_line con sus respectivas métricas medianas: Las métricas de promedio y mediana de tiempo de arribo para las categorías "rojo", "amarillo", "visitas" y moli_medico_on_line muestran una correlación de 1.0. Esto indica que son prácticamente la misma medida o están directamente relacionadas en el *dataset*. En el caso de total_cursos y cant_moli_medico_on_line, muestran una fuerte correlación positiva de 0.89. Un mayor número de cursos se asocia fuertemente con una mayor cantidad de atención médica en línea. Para el caso de laborable y prom_arribo_min_moli_medico_on_line hay una correlación de 0.89, por lo que los días laborables están fuertemente asociados con tiempos de llegada promedio más largos para el médico en línea MOLI.

También `consultas_guardia` y `total_cursos` muestran una alta correlación de 0.71. Un aumento en las consultas de guardia se asocia con un aumento en los cursos totales. `feriado` y `consultas_guardia / total_cursos`: `feriado` se correlaciona positivamente con `consultas_guardia` (0.74) y `total_cursos` (0.68). Esto sugiere que, en promedio, hay más actividad de consultas y cursos durante los días feriados. `laborable` y `consultas_guardia / total_cursos`: También se observa una fuerte correlación positiva entre `laborable` y `consultas_guardia` (0.74) y `total_cursos` (0.68). Esto, junto con la observación de `feriado`, indica que tanto los días laborables como los feriados pueden tener picos de actividad, lo que podría depender de la naturaleza específica de las consultas y cursos. `cant_visitas` y `consultas_guardia / total_cursos` tienen fuertes correlaciones positivas de 0.56 y 0.68, respectivamente. Más visitas se relacionan con más consultas de guardia y más cursos. `cant_moli_medico_on_line` y `cant_visitas`: correlación positiva de 0.49. `cant_rojo` con `med_arribo_min_amarillo` (0.81) y `med_arribo_min_visitas` (0.77): Estas correlaciones cruzadas son notablemente altas, sugiriendo interdependencias entre la cantidad de eventos "rojo" y los tiempos de llegada medianos para "amarillo" y "visitas".

Entre las que presentan correlaciones moderadas/bajas, `temp_min` y `temp_max`: están correlacionadas positivamente (0.49) como es de esperar. Ambas tienen correlaciones generalmente débiles con las métricas de actividad. No obstante, los gráficos muestran que `cant_socios` y `consultas_guardia` pueden variar en ciertos rangos de temperatura. También `cant_socios` muestra correlaciones positivas bajas a moderadas con varias métricas como `total_cursos`, `consultas_guardia`, `internaciones`, `sum_reclamos` y `cant_moli_medico_on_line`. Esto indica que una mayor base de socios se asocia con un uso ligeramente mayor de los servicios. `prom_edad_soc` tiene correlaciones generalmente muy débiles con la mayoría de las variables. Los gráficos de dispersión sugieren tendencias ligeramente crecientes con `consultas_guardia` y `total_cursos`.

Las variables de tipo acumuladoras (`reclamos`, `cambios_color`, `rechazos_internacion`) generalmente están correlacionadas positivamente entre sí y con las variables de contador (`rojo`, `amarillo`, `visitas`, `moli_medico_on_line`), así como con `consultas_guardia` y `total_cursos`. Esto sugiere que los procesos administrativos o las incidencias (`reclamos`, `rechazos`) aumentan con la actividad general.

Se detecta un impacto del día de la semana dado que, aunque la variable `dia_semana` tiene una distribución uniforme y correlaciones débiles con otras variables en la matriz, los gráficos individuales de series temporales por día de la semana (páginas 69-77) muestran variaciones en métricas como `cant_moli_medico_on_line` y `prom_arribo_min_moli_medico_on_line`. Esto sugiere patrones diarios específicos que una correlación lineal general no captura completamente.

También hay un impacto de feriados y días laborables. Los promedios por feriado (cuando es 1) y por laborable (cuando es 1) muestran valores promedio más altos para casi todas las métricas de actividad en comparación con cuando son 0. La correlación negativa de -0.3 entre feriado y laborable es esperada, ya que son categorías opuestas. El hecho de que ambos tengan correlaciones positivas con las métricas de actividad puede indicar que la actividad en el *dataset* es alta tanto en días de semana regulares como en ciertos días festivos que pueden implicar eventos especiales o mayor demanda de servicios, o que los datos son agregaciones que muestran esta tendencia.

Conclusiones sobre el *dataset*

1. **La actividad general está interconectada:** Existe una fuerte interrelación entre las diferentes métricas de actividad, como `consultas_guardia`, `total_cursos`, `internaciones` y la cantidad de eventos rojo, amarillo, visitas y `moli_medico_on_line`. Un aumento en una métrica de actividad suele reflejarse en otras.
2. **Influencia de la atención médica en línea (MOLI):** La cantidad de atenciones de médico en línea (`cant_moli_medico_on_line`) es un factor significativo, con una fuerte correlación con los `total_cursos` y `cant_visitas`, y sus propios tiempos de llegada. Esto sugiere que MOLI es un canal importante de interacción y actividad.
3. **Patrones temporales complejos:** Si bien las temperaturas (`temp_min`, `temp_max`) tienen una influencia limitada en las correlaciones generales, es posible que existan patrones de actividad específicos que dependan de rangos de temperatura o eventos temporales, como se insinúa en los gráficos de dispersión. Los días de la semana también tienen un impacto en los volúmenes de actividad, aunque no se manifieste como una correlación lineal fuerte con el número de día.
4. **Gestión administrativa y de incidencias:** Las métricas relacionadas con `sum_reclamos`, `sum_cambios_color` y `sum_rechazos_internacion` tienden a aumentar con la actividad general, lo que podría indicar que a mayor volumen de interacciones, mayor es la cantidad de incidencias o procesos administrativos a gestionar.
5. **Redundancia de datos:** Las correlaciones de 1.0 entre las métricas de promedio y mediana para los tiempos de arribo de "rojo", "amarillo", "visitas" y `moli_medico_on_line`, así como entre `prom_t_internacion_min` y `mediana_t_internacion_min`, sugieren que estas parejas de variables podrían ser representaciones idénticas o casi idénticas de la misma información, lo que podría simplificar análisis futuros al elegir solo una de ellas.
6. **Concentración de eventos:** El sesgo positivo predominante en muchas variables de recuento y tiempo de llegada indica que la mayoría de los días o períodos tienen niveles de actividad bajos a moderados, con un número menor de eventos o tiempos

excepcionalmente altos. Esto es importante para comprender la variabilidad y la ocurrencia de picos.

7. **Días de alta actividad:** Tanto los días feriados como los días laborables están asociados con un aumento en el promedio de varias métricas de actividad. Esto podría deberse a la naturaleza de los servicios o actividades, donde la demanda es alta tanto en días regulares como en ciertos días especiales.

Cabe tener en cuenta que este análisis proporciona una comprensión inicial de las relaciones dentro del *dataset*, destacando las variables más influyentes y los patrones de comportamiento observados.

Asimismo, la librería presentó cuatro alertas: la variable *dia* contiene valores únicos, lo cual no representa ningún inconveniente al tratarse de una serie temporal, las variables *feriado* y *laborable* contienen una gran cantidad de valores en cero, lo que tampoco es problemático, puesto que ambas son variables dicotómicas. Por último, también se subraya la presencia de mayor cantidad de ceros en la variable *sum_rechazos_internacion*, lo que incluso es positivo, debido a la naturaleza de esta, los intentos de internación tuvieron un desenlace positivo, no rechazando este proceso cuando se ameritara.

AutoViz

Tal como describe Seshadri (2020), esta herramienta se puede utilizar para visualizar automáticamente cualquier conjunto de datos, de cualquier tamaño, con una sola línea de código. El autor destaca de esta su facilidad de uso, velocidad, escalabilidad, automatización, personalización y evaluación de calidad de datos. También menciona que admite distintos tipos de formatos del conjunto de datos, sea en *.csv*, *.txt*, *.json*, o bien un *dataframe* de pandas. Por último, comenta que puede utilizarse como entrada un conjunto de datos con o sin una variable objetivo y puede generar gráficos interactivos que se guardarán en un HTML, en un directorio personalizado.

Para el presente desarrollo, y debido a que la mayoría de las variables comparten un mismo tipo de dato, es decir, son numéricas, por lo que la interacción entre estas genera una gran cantidad de visualizaciones. Atento a lo expuesto, se generó un reporte en formato *.pdf* con descarga automática y se alojó en <https://drive.google.com/file/d/1weP24V-LMtvfRjtFLPo2-NaqKVS7aNSp/view?usp=sharing>. En el caso de estudio la herramienta arrojó un informe permite ver que hay una alta interconexión y dinamismo en los niveles de actividad general dado que existe una fuerte interrelación entre diversas métricas de actividad dentro del *dataset*, lo que sugiere que un aumento en una métrica a menudo se correlaciona con aumentos en otras, indicando periodos de mayor "actividad" general. Por ejemplo: *total_cursos* muestra una correlación de Pearson muy

fuerte con `cant_moli_medico_on_line` (0.95), y también con `sum_cambios_color` (0.72) y `sum_reclamos` (0.66). `consultas_guardia` está fuertemente correlacionada con `cant_visitas` (0.62), `total_cursos` (0.59), `sum_reclamos` (0.57) y `sum_rechazos_internacion` (0.50). Además las métricas de `sum_reclamos` y `sum_rechazos_internacion` están positivamente correlacionadas entre sí (0.52), y ambas aumentan con la actividad general (`total_cursos`, `consultas_guardia`, `cant_moli_medico_on_line`).

Esto indica que la demanda de servicios (consultas, cursos, atenciones en línea) y las actividades administrativas asociadas (reclamos, cambios de color, rechazos de internación) suelen comportarse de manera conjunta, implicando que los picos de actividad en un área a menudo se reflejan en otras.

También muestra marcados patrones temporales en la demanda de servicios: Las variables temporales tienen una influencia significativa en los niveles de actividad. Esto se observa claramente en las asociaciones categóricas (razón de correlación): `dia_semana` presenta altas razones de correlación con `consultas_guardia` (0.64), `sum_reclamos` (0.45), `sum_rechazos_internacion` (0.45), `cant_visitas` (0.40) y `total_cursos` (0.37). De manera similar, `laborable` (indicando si es un día laborable) muestra altas razones de correlación con `consultas_guardia` (0.62), `sum_reclamos` (0.48), `sum_rechazos_internacion` (0.44) y `total_cursos` (0.36). Aunque en menor medida, la variable `feriado` también tiene asociaciones con `consultas_guardia` (0.20) y `sum_reclamos` (0.18).

Estas asociaciones temporales son cruciales para la planificación operativa y la gestión de recursos, ya que la demanda varía de manera predecible según el día de la semana y si es un día laborable o festivo.

Hay una presencia de eventos anómalos y necesidad de análisis detallado en tiempos de respuesta puesto que muchas variables numéricas, especialmente las relacionadas con los tiempos de llegada y recuentos, exhiben un sesgo positivo alto, lo que indica la ocurrencia de valores extremos o atípicos que se extienden significativamente a la derecha de la distribución. `med_arribo_min_rojo` tiene un sesgo de 18.8, `cant_moli_medico_on_line` tiene un sesgo de 3.85. `total_cursos` tiene un sesgo de 2.73 y, por otro lado, `prom_t_internacion_min` presenta un sesgo negativo considerable (-5.84).

Estos sesgos sugieren que, si bien la mayoría de los eventos o tiempos pueden ser moderados, hay una minoría de casos que experimentan demoras o volúmenes excepcionalmente altos o bajos, lo cual es crítico para la calidad del servicio y la gestión de la capacidad.

Hay diferencias en las métricas de tiempo de arribo y tiempo de internación. A diferencia de una suposición de redundancia total, el análisis revela que las métricas de promedio y mediana para

los tiempos de arribo no siempre son idénticas. `prom_arribo_min_visitas` y `med_arribo_min_visitas` tienen una correlación de Pearson alta (0.84). `prom_arribo_min_moli_medico_on_line` y `med_arribo_min_moli_medico_on_line` también muestran una correlación fuerte (0.82). Sin embargo, para "rojo" y "amarillo", estas correlaciones son más moderadas (`prom_arribo_min_rojo` con `med_arribo_min_rojo` 0.33; `prom_arribo_min_amarillo` con `med_arribo_min_amarillo` 0.48). Notablemente, `prom_t_internacion_min` y `mediana_t_internacion_min` tienen una correlación de Pearson muy baja (0.09).

Esto indica que el promedio y la mediana, especialmente para los tiempos de internación, capturan aspectos diferentes de la distribución, y el uso de ambos es valioso para comprender la variabilidad y la centralidad de estas métricas, no siendo medidas redundantes en todos los casos.

Se presenta una relación no lineal de la edad promedio de los socios con la actividad. Aunque la correlación de Pearson para `prom_edad_soc` con muchas variables numéricas es débil, la razón de correlación (que captura asociaciones no lineales) es notablemente más alta para varias métricas de actividad: `cant_amarillo` (0.44), `cant_socios` (0.44), `total_cursos` (0.39), `cant_visitas` (0.38). Esto sugiere que la edad promedio de los socios influye en el uso de los servicios, pero esta relación no es una simple tendencia lineal. Podría haber segmentos de edad con patrones de uso distintos que requerirían un análisis más profundo o modelos no lineales.

Puede verse un comportamiento inesperado de la cantidad de socios: La variable `cant_socios` muestra correlaciones de Pearson negativas con algunas métricas de actividad: `cant_moli_medico_on_line` (-0.36) y `total_cursos` (-0.31). Esta observación es contraintuitiva, ya que una mayor cantidad de socios podría esperarse que correlacionara con un mayor uso de servicios. Podría indicar que el aumento en la base de socios no se traduce en un aumento proporcional en el uso de ciertos servicios por parte de esos nuevos miembros, o que diferentes cohortes de socios tienen patrones de uso distintos. Esto merece una investigación más profunda para entender la demografía o los tipos de servicios que prefieren los distintos grupos de socios.

Hay un impacto limitado de las variables climáticas. Las temperaturas mínimas y máximas (`temp_min`, `temp_max`) tienen una influencia limitada en la mayoría de las métricas de actividad, mostrando correlaciones de Pearson generalmente débiles o moderadamente negativas: `temp_min` se correlaciona negativamente con `cant_visitas` (-0.32) y `consultas_guardia` (-0.21), mientras que `temp_max` muestra relaciones similares con `cant_visitas` (-0.35) y `consultas_guardia` (-0.25). Aunque existe una ligera tendencia a que la actividad disminuya con temperaturas más altas, el clima no parece ser el factor más dominante en la variación de la demanda general de servicios.

Asimismo, también presenta un resumen rápido de cada variable del conjunto de datos, donde puede observarse el tipo de dato, los porcentajes de valores faltantes y únicos de cada una de ellas, así como el mínimo y máximo.

D-Tale y Dataprep

Se intentó, sin éxito, el uso de las librerías *dtale* y *dataprep*. Con la primera de ellas no fue posible lograr ejecutar el reporte interactivo, y tampoco se logró obtener un *link* hacia una *web* en la cual funcionara la ejecución de este. En el segundo caso, la instalación de la librería en el ambiente generó conflictos con la librería *pandas*, la cual fue necesario utilizar posteriormente, por lo que tampoco se logró incorporar el uso de esta para el presente. Sin embargo, con las tres librerías previas se ha podido obtener una gran cantidad de información de forma rápida y automática.

Ingeniería de Características | *Auto Feature Engineering*

Así como se ha podido automatizar en pocas líneas el análisis exploratorio de datos, también existen herramientas para agregar, transformar y/o generar nuevas características al conjunto de datos, es decir, permite aumentar la dimensionalidad con pocos pasos y de forma masiva. A su vez, también es posible utilizar selectores de las variables de mayor importancia. En este caso, se abordaron la utilización de las librerías *tsfresh*, *featuretools* y *autofeat*.

La estrategia utilizada en este caso fue la creación de variables adicionales, entre estadísticas descriptivas con *tsfresh*, y primitivas y transformaciones de *featuretools*. El paso posterior fue, utilizando el *FeatureSelector* de *autofeat* se redujo la dimensionalidad del *dataframe* de acuerdo con la selección de aquellas que la función consideró más importantes.

TsFresh

De acuerdo con la documentación de Christ y Blue Yonder GmbH (2023-2025) de esta librería, *tsfresh* se utiliza para la ingeniería sistemática de características a partir de series temporales y otros datos secuenciales. Las características extraídas pueden utilizarse para describir las series temporales, es decir, ofrecen nueva información sobre las series temporales y su dinámica. También pueden utilizarse para agrupar series temporales.

En el desarrollo de este trabajo, se obtuvieron 288 características de las variables numéricas del conjunto de datos, del cual, previamente, se excluyó la variable objetivo *internaciones*. Entre las características obtenidas se incluyen la suma, media, mediana, desvío estándar, mínimo, máximo,

largo, valor absoluto del máximo, curtosis y asimetría, entre otros. Finalmente, se mantuvo la variable día por tratarse de una serie temporal, y no perder la referencia de la fecha.

FeatureTools

Feature Labs, Inc. (2022) menciona que `featuretools` es un marco para realizar ingeniería de características automatizada. Destaca por transformar conjuntos de datos temporales y relacionales en matrices de características para el aprendizaje automático. Semejante al caso anterior, permite la creación de un nuevo conjunto de datos con las características extraídas del conjunto con el que se está trabajando. La principal diferencia con `tsfresh` es que, también, permite la transformación primitiva de características temporales, es decir, la extracción de datos puntuales de fechas, como ser el día, mes, año, diferencia con algún período, entre otros.

En el desarrollo de este trabajo, se continuó con el conjunto creado con `tsfresh` y se procesó, con `featuretools` las suma, media, mínimo, máximo y desvío estándar de cada variable. Además, se le agregó la media acumulada y suma acumulada de cada característica como novedades aportadas por esta librería. Una vez procesado, el conjunto de datos finalizó con 858 variables. Finalmente, es dable destacar que, con pocas líneas en ambas librerías, se ha podido realizar la creación de una numerosa cantidad de variables de forma automatizada.

AutoFeat

Tal como en los casos anteriores, la documentación de la librería, que se ubica en Andra Ionescu (2024), determina que `autofeat` es una herramienta basada en *Python* que permite mejorar una tabla base añadiendo nuevas características con el objetivo de aumentar significativamente la precisión predictiva de un modelo de aprendizaje automático. Además, emplea estrategias de selección de características basadas en heurísticas para eliminar características redundantes o irrelevantes, por lo que mejora notablemente la eficiencia y la precisión de las operaciones de aprendizaje automático posteriores.

Como fuera mencionado anteriormente, para el presente trabajo, esta librería se utilizó, principalmente, para reducir la dimensionalidad del conjunto de datos aumentado previamente. En primer lugar, a partir del conjunto resultante del procesamiento de `tsfresh` y `featuretools` a este se lo conjugó con el conjunto de datos original, resultando en un conjunto de 884 características. Posteriormente, se realizó la separación de los datos para entrenamiento y prueba de los modelos que se le pudieran aplicar. El rango de datos de entrenamiento se compuso entre el 01/01/2022 y el 28/02/2025, y para prueba se resguardaron los datos de marzo de 2025. Seguido a ello, se realizó una primera reducción de dimensionalidad a partir de la eliminación de variables con baja varianza.

A continuación, con la aplicación de autofeat se seleccionaron 50 características, resultando una reducción del 94,3%. Incluyendo la variable que contiene las fechas, los conjuntos de entrenamiento y *test* finalizaron con 51 características. En la Tabla 1 es posible observar estadísticas descriptivas de las variables seleccionadas.

Tabla 1
Estadísticas descriptivas de las variables numéricas seleccionadas

	count	mean	std	min	25%	50%	75%	max
laborable	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
consultas_guardia	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
total_cursos	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000
sum_reclamos	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
sum_rechazos_internacion	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
cant_rojo	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_amarillo	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_amarillo_sum_values	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_amarillo_mean	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_amarillo_median	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_amarillo_minimum	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_amarillo_maximum	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_amarillo_absolute_maximum	1,155.000	20.538	6.035	3.000	16.000	20.000	24.000	44.000
cant_rojo_sum_values	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_rojo_mean	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_rojo_median	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_rojo_minimum	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_rojo_maximum	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_rojo_absolute_maximum	1,155.000	10.501	3.732	1.000	8.000	10.000	13.000	26.000
cant_visitas_absolute_maximum	1,155.000	23.582	14.226	1.000	13.000	21.000	30.000	91.000
consultas_guardia_sum_values	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
consultas_guardia_mean	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
consultas_guardia_median	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
consultas_guardia_minimum	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
consultas_guardia_maximum	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
consultas_guardia_absolute_maximum	1,155.000	407.051	78.899	164.000	349.000	405.000	464.500	619.000
laborable_sum_values	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
laborable_mean	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
laborable_median	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
laborable_minimum	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
laborable_maximum	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
laborable_absolute_maximum	1,155.000	0.677	0.468	0.000	0.000	1.000	1.000	1.000
sum_rechazos_internacion_sum_values	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
sum_rechazos_internacion_mean	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
sum_rechazos_internacion_median	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
sum_rechazos_internacion_minimum	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
sum_rechazos_internacion_maximum	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
sum_rechazos_internacion_absolute_maximum	1,155.000	32.505	33.168	0.000	8.000	21.000	48.500	217.000
sum_reclamos_sum_values	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
sum_reclamos_mean	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
sum_reclamos_median	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
sum_reclamos_minimum	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
sum_reclamos_maximum	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
sum_reclamos_absolute_maximum	1,155.000	13.175	8.609	0.000	7.000	11.000	18.000	57.000
total_cursos_sum_values	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000
total_cursos_mean	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000
total_cursos_median	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000
total_cursos_minimum	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000
total_cursos_maximum	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000
total_cursos_absolute_maximum	1,155.000	159.164	67.286	51.000	117.000	149.000	185.000	670.000

AutoClean

Hemos puesto en consideración el uso de AutoClean a lo largo del desarrollo, considerando sus pros y contras. Teniendo en cuenta la preponderancia de la etapa de limpieza de datos en el desempeño de los modelos, la elección entre un enfoque manual y uno automatizado se vuelve un aspecto crítico dentro de todo pipeline *de machine learning*.

El trabajo de Li et al. (2021) determina que herramientas automatizadas como AutoClean permiten acelerar el proceso, reducir la carga operativa y garantizar cierta estandarización en tareas repetitivas como imputación de valores faltantes, detección de *outliers* o normalización de variables. Sin embargo, otros estudios, como el de Côté et al. (2024), señalan que la automatización puede introducir riesgos, como la pérdida de información contextual o la aplicación de transformaciones que no reflejan reglas de negocio específicas, lo que puede impactar en la interpretabilidad del modelo.

En este caso, se ha decidido contrastar ambos métodos, presentando ello en la Tabla 2: por un lado, el preprocesamiento manual, el cual permitió definir criterios propios (como categorización de edad, rangos de antigüedad, variables *dummies* de estado civil y empleo, o la manipulación cuidadosa de datos temporales); y por otro, la exploración del uso de AutoClean como herramienta de referencia. Este análisis comparativo busca fundamentar la decisión de mantener control sobre la limpieza de datos en aquellas variables sensibles al dominio del problema, sin dejar de reconocer las ventajas que ofrece la automatización en escenarios de alta dimensionalidad o *datasets* con ruidos poco estructurados.

Tabla 2

Selección de características | Procesamiento manual o uso de librería AutoClean

Criterio	Preprocesamiento Manual	AutoClean
Control sobre el dominio	Alto: definimos variables de negocio (edad en rangos, antigüedad en meses, estado civil con <i>dummies</i> , etc.) de forma consciente y alineada al problema.	Bajo: aplica transformaciones genéricas sin contemplar reglas específicas del dominio.
Flexibilidad	Muy alta: podemos decidir cuándo imputar, cómo tratar <i>outliers</i> o si mantener valores nulos con significado.	Limitada: AutoClean decide automáticamente la estrategia de imputación, normalización y detección de <i>outliers</i> .
Tiempo y esfuerzo	Mayor costo de tiempo y conocimiento técnico, ya que implica revisar <i>dataset</i> , codificar reglas y <i>testear</i> cada paso.	Ahorro de tiempo: en pocas líneas realiza limpieza, estandarización y preparación de <i>features</i> .

Criterio	Preprocesamiento Manual	AutoClean
Transparencia / Interpretabilidad	Total: cada paso es explícito y documentado en el código, lo que facilita explicar al cliente cómo se construyó la ABT.	Parcial: la lógica interna es más difícil de auditar y explicar a usuarios no técnicos.
Reproducibilidad	Muy alta: el <i>pipeline</i> manual puede versionarse en <i>Git</i> y replicarse exactamente.	Alta: AutoClean también es reproducible, pero depende de la versión de la librería y sus defaults.
Robustez ante ruidos no previstos	Media: si aparece un tipo de error no contemplado en el pipeline manual, puede romper la ejecución.	Alta: al estar diseñado para escenarios genéricos, AutoClean maneja bien anomalías inesperadas.
Escalabilidad	Menos eficiente en <i>datasets</i> muy grandes o con alta dimensionalidad (requiere mucho tiempo de ajuste manual).	Muy eficiente para grandes volúmenes y alta dimensionalidad, con mínimo esfuerzo adicional.
Riesgo de sobre procesamiento	Bajo: aplicamos solo lo necesario para el modelo y el negocio.	Alto: al aplicar reglas genéricas puede eliminar variabilidad o patrones útiles para el modelo.
Adaptación a temporalidad	Excelente: la separación por día y los cortes de entrenamiento/prueba fueron hechos con criterio temporal.	Limitada: AutoClean no tiene mecanismos nativos para respetar la estructura temporal de los datos.

El preprocesamiento manual resulta ser la mejor alternativa en nuestro caso. Dado que el volumen de datos que manejamos no es tan grande, no representa una carga significativa implementar reglas personalizadas y específicas del dominio. Esto nos permite garantizar control total, interpretabilidad y alineación con el problema de negocio (consultas de guardia con estructura temporal).

Si bien AutoClean aporta rapidez y robustez en entornos de gran escala o cuando los datos presentan muchas anomalías, en este proyecto su uso podría implicar sobre procesar la información o aplicar transformaciones genéricas que no necesariamente respeten la lógica del dominio.

Selección de Modelo | *AutoML*

Así como sucede en el análisis exploratorio de los datos y en la ingeniería de características, existen librerías que permiten el modelado automático y veloz, entrenamiento y prueba del conjunto en este desarrollo. En el presente, se han seleccionado los paquetes h2o, PyCaret, Auto-PyTorch, Auto Scikit-Learn y AutoKeras. Por disponibilidad de tiempo, sólo se ha podido evaluar la utilización de h2o, PyCaret y AutoKeras.

H2O

La documentación oficial de esta librería, H2O.ai (2025), define que esta es un software basado en Java para modelado de datos y computación en general, donde su propósito principal es ser un motor de procesamiento distribuido, paralelo y en memoria. El módulo H2O de Python es una herramienta para procesar modelos rápidamente, procesar datos y crear aplicaciones en un entorno rápido y escalable. Este paquete permite desde la apertura de un servidor *web* para hacer el seguimiento del proceso, el procesamiento de distintos modelos ya contenidos dentro de la librería y ordenarlos de acuerdo con la minimización de los errores que tengan y ejecutar predicciones con el modelo elegido. Asimismo, es posible ejecutar, dentro de ese ambiente, una técnica de *grid search* para probar distintas combinaciones de hiperparámetros del modelo elegido e intentar mejorar aún más su rendimiento.

Durante este proceso, se seleccionó la utilización de hasta 15 modelos distintos y un máximo de ejecución de todo el proceso de 10 minutos. La librería entrena distintos algoritmos, ajuste hiperparámetros y realiza *cross validation* para predecir la variable objetivo. Finalmente, se presentan los distintos modelos ordenados de acuerdo con el que logra la mayor reducción de la raíz del error cuadrático medio. En este caso fue un *Stacked Ensemble*, el cual ponderó los resultados de los distintos modelos probados y los combinó para obtener un mejor resultado, de los cuales solo eligió el mejor de cada “familia” de modelos. Las métricas de este modelo al aplicarlo en el conjunto de prueba se presentan en la Tabla 3.

Tabla 3

Métricas en el Test Set del mejor modelo seleccionado por H2O AutoML

Métrica	Valor (Test Set)
MSE	40.73
RMSE	6.38
MAE	5.39
R ² (R-squared)	0.288
Desviación Residual Media	40.73

PyCaret

De igual forma que en el caso anterior, esta librería permite automatizar los flujos de trabajo de aprendizaje automático. Tal como destaca Ali (2020), es una herramienta integral de aprendizaje automático y gestión de modelos que acelera exponencialmente el ciclo experimental y aumenta la productividad, debido a ser *low-code*, lo que permite reemplazar cientos de líneas de código con solo unas pocas, y que los experimentos sean exponencialmente rápidos y eficientes.

En el caso actual, debido a la incompatibilidad de la librería con el versionado de *Python* con el que trabaja *Google Colab*, en la sección de esta librería se generó un ambiente virtual exclusivo para la ejecución de sus comandos. En este se utilizaron los modelos que contiene la librería, eligiendo aquellos para la tarea de regresión. El mejor modelo elegido por *PyCaret* fue un *Huber Regressor*, y sus métricas se presentan en la Tabla 4.

Tabla 4

Métricas del mejor modelo seleccionado por PyCaret

Modelo	MSE	RMSE	MAE	R ²
<i>Huber Regressor</i>	40.81	6.38	5.16	0.168

AutoKeras

AutoKeras es una librería de *Python* de *AutoML* enfocada en *deep learning* que automatiza el diseño de modelos de redes neuronales para distintos tipos de datos, como imágenes, texto y datos tabulares. A partir de un dataset y una variable objetivo, *AutoKeras* realiza búsqueda automática de arquitecturas de red neuronal, ajuste de hiperparámetros y preprocesamiento de datos, seleccionando el modelo que obtiene mejor desempeño según la métrica indicada. Su objetivo es simplificar el desarrollo de modelos de machine learning complejos sin necesidad de conocimientos profundos en redes neuronales, y permite exportar el modelo final como un modelo *Keras* estándar para entrenamientos adicionales o despliegue.

A partir del archivo de sumario se puede ver que el modelo recomendado por *AutoKeras* es una red neuronal totalmente conectada (*fully connected*) diseñada para resolver un problema de regresión, en este caso la predicción de la variable internaciones. La arquitectura comienza con una capa de entrada que normaliza los datos a formato *float32*, seguida de dos capas densas intermedias con activación *ReLU*, lo que permite capturar relaciones no lineales entre las variables. Estas capas están acompañadas de múltiples bloques de *Dropout*, una técnica de regularización que apaga aleatoriamente neuronas durante el entrenamiento para evitar el sobreajuste y mejorar la capacidad de generalización del modelo. Finalmente, la red termina en una capa de salida densa con activación lineal que produce un único valor numérico como predicción.

En términos de complejidad, el modelo tiene alrededor de 122 mil parámetros, de los cuales unos 40 mil son entrenables, lo que lo convierte en una red relativamente compacta dentro del espectro de modelos de *deep learning*. Esta magnitud es adecuada para trabajar con datos tabulares, ya que no es tan pequeña como para quedarse corta en capacidad de representación, ni tan grande como para sobreajustar con facilidad, especialmente si el dataset no es muy grande. El uso repetido de *Dropout* refuerza esta intención de controlar el sobreentrenamiento, señal de que *AutoKeras* priorizó un modelo balanceado más que uno extremadamente complejo.

El desempeño reportado es un RMSE en test de 6.37, lo cual significa que en promedio el modelo se equivoca alrededor de 6 unidades al predecir el valor real de internaciones. La calidad de este resultado depende de la escala y variabilidad del dataset: si los valores de internaciones son bajos, el error puede ser alto en proporción; pero si los valores típicos están en decenas o centenas, puede considerarse un error razonable. En cualquier caso, la interpretación más importante es que el modelo seleccionado por *AutoKeras* fue el que mejor equilibró precisión y robustez entre todas las configuraciones probadas, constituyendo una buena base inicial que podría luego ajustarse o combinarse con otras técnicas de optimización de hiperparámetros para mejorar aún más el rendimiento.

Tabla 5

Métricas del mejor modelo seleccionado por AutoKeras

Modelo	MSE	RMSE	MAE	R ²
<i>Custom Best_AutoKeras_Model</i>	40.58	6.37	5.09	0.14

Optimización de Hiperparámetros | *AutoHPO*

Finalmente, y como sucedió en las secciones anteriores, también existen automatizaciones en la optimización de los distintos hiperparámetros de los modelos elegidos, utilizados o autoseleccionados por alguna herramienta de apoyo. Entre las distintas alternativas utilizadas en este trabajo se destacan *grid search* con la librería *h2o*, optimización bayesiana con *optuna* y *random search* con un método más eficiente a través de *optuna hyperband*.

En los tres casos, se partió de los mejores modelos obtenidos con *h2o automl*, se optimizaron los hiperparámetros, y se construyó un *ensemble* para equiparar con el mejor modelo seleccionado inicialmente por *h2o*.

H2O | *Grid Search*

De acuerdo con la documentación de esta librería, que se expresa en H2O.ai (2025), existe la posibilidad de utilizar una búsqueda cartesiana según los valores que se designen manualmente para cada hiperparámetro a optimizar, o bien, determinar únicamente un espacio de búsqueda y que la librería determine los valores cartesianos dentro de ese espacio. En ambos sentidos, existe la posibilidad de detener la búsqueda por un número máximo de modelos, un número máximo de segundos o alguna métrica de rendimiento. Luego de ello, es posible consultar el resultado ordenado por alguna de estas métricas.

En el marco de este trabajo, se realizó una optimización y se resguardaron los resultados para poder importarlos en iteraciones posteriores y evitar repetir el tiempo de trabajo de forma constante. Asimismo, como fuera mencionado anteriormente, a partir de los tres modelos optimizados, se creó un *ensemble* y se obtuvieron las métricas de los cuatro modelos, las cuales se presentan en la Tabla 6, en el conjunto de pruebas.

Tabla 6
Métricas en Test Set — Modelos optimizados con Grid Search

Modelo	MSE	RMSE	MAE	R ²
GBM GS	40.31	6.35	5.43	0.2954
DRF GS	51.01	7.14	5.89	0.1084
XGB GS	39.32	6.27	4.82	0.3126
Ensemble GS	40.38	6.35	5.42	0.2941

Optuna | Optimización Bayesiana

Tal como desarrollan Bais (2025) y Frazier (2018), la optimización bayesiana, a diferencia de una búsqueda aleatoria o cartesiana, así como también de otras que pueden entenderse de “caja negra”, considera los resultados de evaluaciones anteriores al seleccionar el siguiente conjunto de hiperparámetros, por lo que se centra en las áreas del espacio de búsqueda con mayor probabilidad de optimizar el rendimiento del modelo. Al utilizar una regresión con procesos gaussianos, se proporciona una distribución de probabilidad posterior sobre los posibles valores de la función en cualquier punto. Esta distribución se actualiza cada vez que se observa un nuevo punto. Finalmente, limita el número de llamadas a la función objetivo, a la vez que garantiza que las combinaciones de hiperparámetros evaluadas tengan cada vez mayor probabilidad de producir un modelo óptimo.

Optuna, de acuerdo con Akiba et al (2019), es un *framework* de *software* para la optimización automática de hiperparámetros, especialmente diseñado para el aprendizaje automático. La librería

proporciona herramientas para el muestreo de hiperparámetros, definiciones de los espacios de búsqueda de estos, una diversa cantidad de algoritmos para optimizar los hiperparámetros y la facilidad de tareas en paralelo.

Al igual que en el acápite anterior, se determinó un espacio a través del cual el optimizador realizó la búsqueda bayesiana y obtuvo, también, los mejores hiperparámetros, que fueron probados, para el mejor *gradient boosting machine*, el mejor *distributed random forest* y el mejor *xgboost*. Además, se ensamblaron estos modelos creando una cuarta alternativa. Las métricas de todos estos modelos se presentan en la Tabla 7.

Tabla 7
Métricas en Test Set — Modelos optimizados con Optimización Bayesiana

Modelo	MSE	RMSE	MAE	R ²
GBM BO	32.42	5.69	4.74	0.4334
DRF BO	51.99	7.21	6.01	0.0912
XGB BO	37.25	6.10	4.95	0.3489
Ensemble BO	36.10	6.01	5.15	0.3690

Optuna | Random Search Hyperband

De igual forma que en el paso anterior, la librería optuna permite la optimización mediante una búsqueda aleatoria, en este caso potenciada por *hyperband* y con muestreo por *TPE Sampler*.

Random search funciona de forma similar a *grid search*, pero el usuario determina únicamente el espacio de búsqueda, las combinaciones de hiperparámetros serán seleccionadas por el optimizador.

Hyperband es, de acuerdo con lo expresado por Li et. al (2018), una metodología que busca acelerar la búsqueda aleatoria a través de una asignación adaptativa de recursos y una estrategia de detención temprana. La diferenciación principal viene dada en su enfoque para explorar el espacio de búsqueda. *Hyperband* acelera la búsqueda aleatoria al no entrenar todas las configuraciones hasta el final. En cambio, asigna recursos de manera adaptativa, descartando rápidamente las configuraciones de bajo rendimiento y dedicando más recursos a las más prometedoras.

Relevando lo desarrollado por Akiba et al (2019), *TPE Sampler* es un algoritmo denominado Estimador Parzen estructurado en árbol, el cual, también, es un optimizador bayesiano, por lo que utiliza modelos de probabilidad para aprender de los resultados pasados y decidir de forma inteligente dónde probar a continuación, lo que lo hace más eficiente que los métodos de búsqueda exhaustiva.

En este último método de optimización, se combinaron los tres conceptos anteriores para aumentar las alternativas de modelos para pronosticar la variable objetivo. Las métricas de los cuatro modelos obtenidos, al igual que en los pasos anteriores, se presentan en la Tabla 8.

Tabla 8
Métricas en Test Set — Modelos optimizados con Random Search

Modelo	MSE	RMSE	MAE	R ²
GBM RS	31.13	5.58	4.73	0.4558
DRF RS	49.13	7.01	5.78	0.1411
XGB RS	36.18	6.02	4.89	0.3675
Ensemble RS	35.53	5.96	5.08	0.3789

Evaluación Final de los modelos

Una vez hechas las pruebas con los distintos modelos relevados anteriormente, se presentan, en Tabla 9, la comparación de las métricas de cada uno de ellos. Sobre ello es importante recordar que los modelos utilizados fueron el *Stacked Ensemble* generado por la librería H2O, el *Huber Regressor* seleccionado por *PyCaret*, el seleccionado por *AutoKeras*, *Custom Best_AutoKeras_Model*, y los cuatro modelos optimizados (GBM, DRF, XGB y su *ensemble* posterior) con *grid search*, optimización bayesiana y *random search*.

Tabla 9
Métricas en Test Set — Comparación entre todos los modelos seleccionados

Model	MSE	RMSE	MAE	R2
H2O Stacked Ensemble	40.56	6.37	5.38	0.2909
Huber Regressor	40.81	6.38	5.16	0.1676
Custom Best_AutoKeras_Model	44.87	6.70	5.30	0.0462
GBM GS	40.31	6.35	5.43	0.2954
DRF GS	51.01	7.14	5.89	0.1084
XGB GS	39.32	6.27	4.82	0.3126
Ensemble GS	40.38	6.35	5.42	0.2941
GBM BO	32.59	5.71	4.71	0.4304
DRF BO	48.69	6.98	5.75	0.1489
XGB BO	38.19	6.18	5.12	0.3324
Ensemble BO	36.18	6.01	5.05	0.3676
GBM RS	31.13	5.58	4.73	0.4558
DRF RS	49.13	7.01	5.78	0.1411
XGB RS	36.18	6.02	4.89	0.3675
Ensemble RS	35.53	5.96	5.08	0.3789

De lo resultante presentado anteriormente, se obtienen conclusiones de gran interés. En primer lugar, se destaca la muy baja explicación que se obtuvo a partir de los datos utilizando las librerías de *PyCaret* y *AutoKeras*, a pesar de que las métricas de lo obtenido con la primera librería se aproximan a la media y mediana de todas las de la tabla anterior. Asimismo, de forma general, se observa que el conjunto de los modelos optimizados con *random search*, potenciado por *hyperband* y TSE *sampler*, superan al resto en capacidad predictiva y precisión.

En función de las distintas métricas, se visualiza que el *gradient boosting machine*, optimizado con la búsqueda aleatoria y muestreo por optimización bayesiana, es aquel realiza la mayor reducción del error cuadrático medio (MSE), el cual penaliza los errores grandes, y la raíz de este error (RMSE), la cual se interpreta en las mismas unidades que la variable objetivo, por lo que puede mencionarse que, este modelo, alcanza un error medio de entre 5 y 6 internaciones por día. Además, es aquel que mejor explica la variabilidad del modelo según lo expuesto en el coeficiente de determinación, a pesar de tratarse de un valor lejano al óptimo, de entre lo probado, es el mejor. Finalmente, puede destacarse que el *gradient boosting machine*, con optimización bayesiana, absorbe mejor el error absoluto medio (MAE), siendo este modelo el más robusto frente a los *outliers*.

Atento a lo expuesto, el mejor desempeño general fue obtenido por el modelo *gradient boosting machine* optimizado con *random search*, *hyperband* y TSE *sampler*.

Automatización del proceso | *MLOps*

MLOps (Machine Learning Operations) se define como un enfoque metodológico que integra principios de ingeniería de software, operaciones de sistemas y aprendizaje automático con el propósito de gestionar de manera sistemática y escalable el ciclo de vida completo de los modelos de *machine learning*. Este enfoque abarca las etapas de ingesta y preparación de datos, entrenamiento y validación de modelos, despliegue en entornos de producción, monitoreo continuo de su desempeño y procesos de reentrenamiento automatizado.

El uso de *MLOps* resulta particularmente relevante en el contexto de *AutoML*, dado que este último se centra en la automatización de la selección de algoritmos, ajuste de hiperparámetros y generación de modelos como propone en Markov (2023). No obstante, la mera automatización del modelado no garantiza por sí misma la reproducibilidad, la trazabilidad ni la sostenibilidad en producción. La incorporación de prácticas de *MLOps* en un flujo de *AutoML* según Symeonidis (2022) aporta:

- **Reproducibilidad:** posibilita reconstruir experimentos y resultados generados por los motores de *AutoML*.

- **Trazabilidad y versionado:** asegura la vinculación entre datos, configuraciones y modelos resultantes.
- **Despliegue controlado:** facilita la transición de los modelos seleccionados hacia entornos productivos mediante pipelines automatizados.
- **Monitoreo y mantenimiento:** permite detectar degradaciones en el desempeño del modelo y activar mecanismos de reentrenamiento.
- **Gobernanza y cumplimiento:** establece mecanismos para el control de calidad, la auditoría y la alineación con normativas.

Por esto, mientras que *AutoML* optimiza la fase de construcción de modelos, *MLOps* proporciona el marco operativo necesario para su gestión integral, asegurando que los modelos generados puedan ser utilizados de manera confiable, escalable y sostenible en entornos reales.

Solución *MLOps*

En línea con lo planteado con Kreuzerberg (2023), una solución de *MLOps* se concibe como una arquitectura integral destinada a sostener de forma continua y controlada el ciclo de vida de los modelos de aprendizaje automático. Esta solución articula diversos componentes especializados, cada uno de los cuales cumple funciones críticas para la reproducibilidad, la escalabilidad y la gobernanza de los sistemas basados en inteligencia artificial.

La solución de *MLOps*, disponible en <https://github.com/georgsmeinung/mlops-pipeline>, concebida de este modo, no se limita a la mera automatización técnica, sino que constituye un marco metodológico para asegurar la calidad, la escalabilidad y la sostenibilidad de los modelos de aprendizaje automático, incluyendo aquellos generados mediante enfoques de *AutoML*. Su adopción permite transformar modelos experimentales en activos productivos gestionados con criterios de ingeniería y gobernanza. Para el pipeline objetivo de este trabajo la solución se implementa de esta manera:

- **Gestión de entornos:** La solución plantea que cada etapa del pipeline se ejecute en un entorno aislado, creado bajo demanda con `conda` o `pip`. Los *scripts* `create_conda_env.py` y `manage_conda_envs.py` permiten crear, listar y borrar entornos de manera reproducible, además de registrar o desregistrar *kernels* de *Jupyter*. Esto asegura que cada notebook corra con la versión exacta de *Python* y las dependencias declaradas en un archivo YAML, evitando conflictos y garantizando trazabilidad.
- **Ejecución de notebooks:** Se utiliza *Papermill* para ejecutar *notebooks* parametrizados. Cada notebook (por ejemplo, `ingest.ipynb`, `train.ipynb`, `evaluate.ipynb`) puede recibir parámetros desde un archivo YAML y producir una salida ejecutada que se guarda

en la carpeta `out/`. Esto permite que el mismo código sirva tanto para exploración interactiva en *Jupyter Lab* como para ejecución automatizada en un *pipeline*, sin duplicar lógica.

- **MLflow:** Actúa como el sistema de *experiment tracking* y gestión de artefactos. Cada etapa del pipeline (ingesta, entrenamiento, evaluación) *loguea* parámetros, métricas y resultados en *MLflow*. Así, se pueden comparar *runs*, auditar configuraciones y recuperar modelos entrenados desde un repositorio centralizado. Para simplificar la puesta en marcha, la solución usa SQLite como *backend* y la carpeta `mlruns/` como almacén de artefactos, aunque puede escalarse a bases y *storages* distribuidos.
- **Orquestación con GoCD:** El pipeline completo se define en *GoCD*, donde cada *stage* representa una etapa del flujo de *MLOps*. Por ejemplo, `setup_env_ingest` crea el entorno para ingesta, y `run_ingest` ejecuta el notebook correspondiente. *GoCD* garantiza la ejecución secuencial o paralela según dependencias, versiona la configuración del pipeline, y facilita la repetición exacta de procesos en distintos agentes. Además, permite conectar con control de versiones (*GitHub*) y con *MLflow*, cerrando el ciclo de CI/CD para ciencia de datos.
- **Experimentación interactiva con Jupyter:** el equipo de *data science* pueden abrir *Jupyter Lab* para explorar datos y prototipar modelos en los mismos entornos que luego correrán en el pipeline automatizado. Gracias al registro de *kernels*, cada notebook puede usar exactamente la versión de *Python* y librerías declaradas para su etapa. Esto reduce la brecha entre exploración manual y ejecución en producción, reforzando la reproducibilidad.

MLFlow

En esta solución, *MLflow* se integra como la capa de trazabilidad y auditoría de todo el pipeline *MLOps*, funcionando tanto en modo interactivo como automatizado a través de *GoCD* y notebooks. Cada etapa del pipeline (*ingest*, *features*, *train*, etc.) puede tener su propio experimento en *MLflow*, o un único experimento con múltiples *runs*:

```
import mlflow

mlflow.set_tracking_uri("http://localhost:5000")
# también puede referencias un archivo file:///path/to/mlruns
mlflow.set_experiment("ingest-experiment")
```

Esto asegura que cada ejecución quede organizada y fácilmente rastreable. Se puede diferenciar cada run con un `run_name`, por ejemplo, el `stage + timestamp`. Dentro de cada *notebook* (o *script Python*), se *loguean* los parámetros de la etapa:

```
mlflow.log_param("stage", "ingest")
mlflow.log_param("python_version", sys.version)
mlflow.log_param("git_commit", "<commit_sha>")
```

Esto incluye información útil para reproducibilidad: versión de *Python*, *commit* de *Git*, parámetros de *notebook*, *paths* de datos, etc. Si la etapa genera métricas (por ejemplo, precisión, RMSE, cantidad de registros procesados, etc.), se registran en *MLflow*:

```
mlflow.log_metric("num_rows_processed", 15000)
mlflow.log_metric("missing_values_percentage", 0.02)
```

Cada run queda asociado a estas métricas, permitiendo comparar ejecuciones. Se pueden guardar archivos generados por la etapa:

- *Notebooks* ejecutados con *papermill* (*_out.ipynb)
- *pip_freeze.txt* o *requirements.txt* del entorno
- Modelos entrenados
- *Plots*, reportes, CSVs intermedios

```
mlflow.log_artifact("out/ingest_out.ipynb")
mlflow.log_artifact("ml_envs/ml_ingest_py38_pip_freeze.txt")
mlflow.log_artifact("models/model_v1.pkl")
```

Esto asegura auditoría completa: todo lo que se genera queda guardado y versionado.

Además, usando la integración con *conda* y *notebooks* usando los *scripts* adaptados a esta herramienta tenemos adaptamos:

- `create_conda_env.py` crea el entorno y lo registra como *kernel Jupyter*.
- `run_notebook.py` ejecuta el *notebook* con *papermill* usando el *kernel* del entorno *Conda*.
- Dentro del *notebook*, se *loguean* los parámetros, métricas y artefactos en *MLflow*.

Ejemplo mínimo en *notebook*:

```
import mlflow, sys, subprocess

mlflow.set_experiment("ingest-experiment")
with mlflow.start_run(run_name="ingest_run_001"):
    mlflow.log_param("python_version", sys.version)
    freeze = subprocess.check_output(
        [sys.executable, "-m", "pip", "freeze"]
    ).decode()
    mlflow.log_text(freeze, "pip_freeze.txt")
    mlflow.log_artifact("out/ingest_out.ipynb")
```

Con esta integración, cada run en GoCD tiene un registro completo en *MLflow*, incluyendo:

- Versiones de librerías (*pip freeze*)
- *Notebook* ejecutado
- Parámetros de entrada
- Métricas
- *Git commit* asociado

Se puede reproducir cualquier run montando el mismo entorno *Conda* (o *virtual env*) y ejecutando el mismo *notebook*. El resultado de lo registrado en MLflow puede ser luego revisado desde la interfaz web que provee la herramienta.

Así, por ejemplo, para el experimento donde usamos un *Random Forest* para la predicción:

The screenshot shows the MLflow web interface at version 3.3.2. The browser address bar shows a URL from ngrok. The interface has a sidebar on the left with 'Experiments' selected. The main content area is titled 'Prediccion_Consultas' and shows a table of runs under the 'Experimental' tab. The table has columns for Model name, Status, Created, Logged from, Source run, and Registered models. One run is visible: 'modelo_random_forest' with a 'Ready' status, created 21 minutes ago, logged from 'colab_kernel_launcher.py', and sourced from 'intelligent-eel-228'.

Model name	Status	Created	Logged from	Source run	Registered models
modelo_random_forest	Ready	21 minutes ago	colab_kernel_launcher.py	intelligent-eel-228	-

Podemos revisar las métricas de este ingresando al modelo:

The screenshot shows the mlflow 3.3.2 web interface. On the left is a sidebar with a menu icon, the mlflow logo, and a '+ New' button. Below this are links for 'Experiments', 'Models', and 'Prompts'. The main area displays the details for a model named 'modelo_random_forest' under the path 'Prediccion_Consultas > Models >'. It has tabs for 'Overview', 'Traces', and 'Artifacts'. The 'Overview' tab is active, showing a table with the following data:

Source run ID	Value
144e0b412e34e09d10d11a002a0673	

Below this table, there are more details:

- Logged from:** colab_kernel_launcher.py
- Datasets used:** -
- Model versions:** -

At the bottom, there are two sections: 'Metrics (1)' and 'Parameters (2)'. The 'Metrics' section has a search bar and a table with one row:

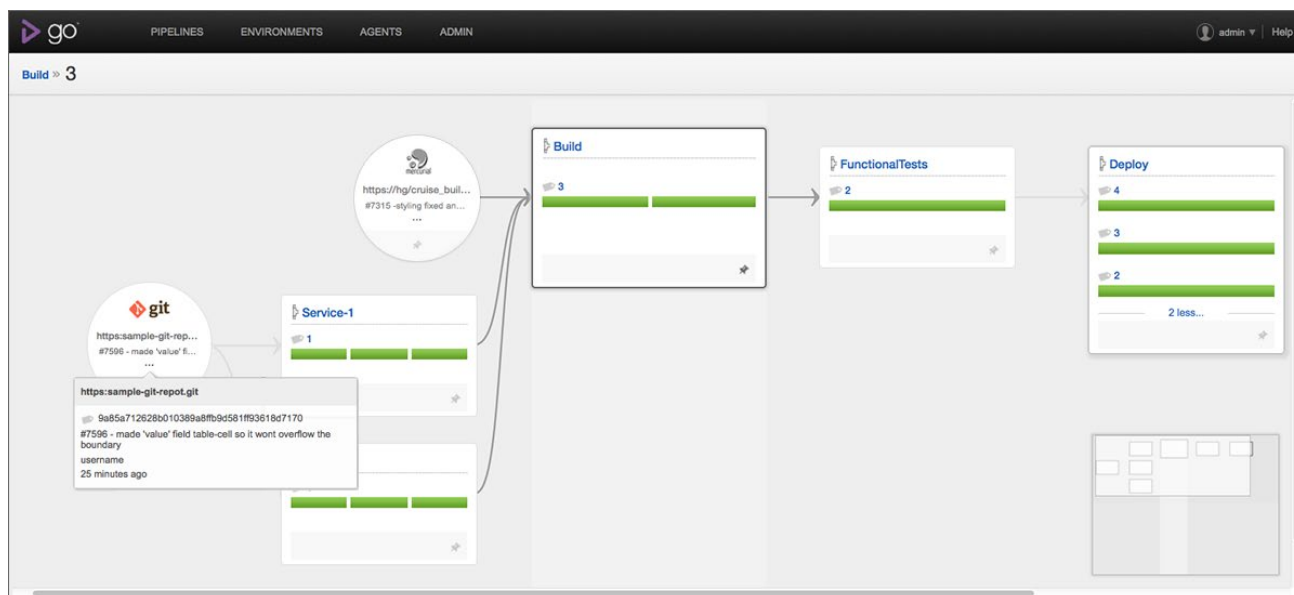
Metric	Dataset	Source run	Value
MSE	-	intelligent-eel...	1528.32779650...

The 'Parameters' section also has a search bar and a table with two rows:

Parameter	Value
n_estimators	100
max_depth	5

GoCD

GoCD es un servidor de integración y entrega continua (CI/CD) de código abierto que ayuda a los equipos de desarrollo a automatizar y gestionar el proceso completo de entrega de *software*. Se diferencia de otras herramientas al centrarse en los *pipelines*, tratándolos como un concepto fundamental para modelar de manera visual y clara las etapas de construcción, prueba y despliegue del código. Esto permite una mejor comprensión de los flujos de trabajo complejos y la detección rápida de problemas.



Para lograr su propósito, GoCD utiliza un servidor que orquesta las tareas y agentes que las ejecutan, siguiendo una arquitectura maestro-esclavo para escalar el entorno. Una de sus herramientas más destacadas es el "Mapa de flujo de valor", que ofrece una visión integral de todas las dependencias del proceso, desde el commit de código hasta su despliegue en producción.

Gracias a esto, los equipos pueden tener una trazabilidad completa de los artefactos y controlar cada fase del ciclo de vida del desarrollo.

GoCD tiene el rol de orquestador de *pipelines* de *MLOps* en la solución planteada. A diferencia de un *scheduler* más lineal, *GoCD* está diseñado para representar flujos de entrega complejos con dependencias explícitas entre etapas. En este caso, cada *notebook* que se ejecuta con *papermill* y entornos *conda* es una etapa en el *pipeline*: ingesta, limpieza, entrenamiento, validación y despliegue. Así, el flujo completo de queda versionado y trazable dentro de un único sistema de CI/CD.

Con *GoCD* se puede configurar cada etapa para que se ejecute en un contenedor o en un agente *conda* específico (ver sección de entornos virtuales), asegurando que las versiones de librerías y dependencias coincidan con lo registrado en *MLflow*. Esto se complementa bien con el mecanismo actual de crear y borrar entornos con *scripts*: *GoCD* simplemente invoca esos *scripts* como parte de la definición del *pipeline*. Además, permite usar plantillas, de modo que la lógica para ejecutar *notebooks* con distintos parámetros puede reutilizarse en varios flujos sin duplicar configuraciones.

Finalmente, *GoCD* facilita la integración con *MLflow* en el plano de trazabilidad. Cada etapa del *pipeline* puede *loguear* automáticamente parámetros, métricas y artifacts en *MLflow*, quedando vinculados al run correspondiente. Así, al navegar en *MLflow* se pueden ver los detalles de la experimentación, y al mismo tiempo, con *GoCD* se puede auditar en qué agente, con qué commit de código y con qué entorno se ejecutó. Esta combinación entrega un nivel de gobernanza y auditabilidad muy superior al de un simple *scheduler* o a correr *notebooks* manualmente.

Problemas de versionado de *Python* | Planteo de solución

El versionado de *Python* suele ser un dolor de cabeza en proyectos de ciencia de datos y *MLOps*, y lo fue en este caso. Diferentes etapas del *pipeline*, librerías o *frameworks* requieren versiones específicas del intérprete y de paquetes que no siempre son compatibles entre sí. Esto lleva a conflictos clásicos: un modelo que corre en *Python* 3.8 no arranca en 3.11, una librería que funciona con *pandas* 1.5 rompe con la 2.x, o una extensión de *Jupyter* que depende de una versión concreta de *ipykernel*. Sin una gestión ordenada, se multiplican los entornos manuales, se pierde trazabilidad y reproducibilidad, y cada miembro del equipo termina con configuraciones diferentes.

La solución planteada para nuestro *pipeline* se basa en *Conda* como gestor central de entornos. *Conda* permite instalar múltiples versiones de *Python* en paralelo y definir entornos aislados donde cada uno contiene exactamente las librerías y dependencias que necesita una etapa del *pipeline*. Además, soporta tanto paquetes *Python* como dependencias de sistema, lo que evita

muchos problemas que `pip` no puede resolver solo. En este esquema, cada etapa del flujo de *MLOps* (ingesta, *features*, entrenamiento, etc.) cuenta con un YAML de especificación que define su *Python* y su *stack* de librerías, lo que garantiza que los resultados sean reproducibles en cualquier máquina.

```
python_version: "3.10"
venv_name: "ml_autoeda_py310"
packages:
  - mlflow==1.29.0
  - papermill>=2.4.0
  - pandas
  - requests
  - autofeat
  - autoviz
  - dask
  - tabulate
  - ydata-profiling
kernel_display_name: "Python (ml_autoeda_py310)"
```

Este enfoque no solo resuelve los conflictos de versiones, sino que además habilita auditoría y trazabilidad completas: cada entorno queda registrado junto con su `conda list` y se puede recrear en cualquier momento. Al integrarlo con *GoCD* y *MLflow*, se logra que cada pipeline cree automáticamente su entorno `conda`, ejecute el *notebook* correspondiente y guarde en *MLflow* tanto los resultados como la especificación del entorno. Así, se combina la flexibilidad de *Conda* con un control riguroso sobre dependencias y versiones, eliminando los problemas clásicos de reproducibilidad.

Conclusiones

En el presente se abordó un desafío estratégico crítico para una empresa de medicina prepaga: la alta variabilidad diaria de las internaciones no programadas, un problema que genera costos elevados debido a las derivaciones externas y la subutilización de recursos en días de baja demanda. La solución propuesta se centró en el desarrollo de un sistema predictivo basado en series temporales capaz de anticipar la demanda futura de internaciones. Metodológicamente, se adoptó un enfoque intensivo en la automatización (*AutoML*), aplicando librerías especializadas lo largo de todo el *pipeline*, incluyendo el Análisis Exploratorio de Datos (*AutoEDA*), la Ingeniería y Selección de Características (*AutoFE*) y la Selección y Optimización de Modelos. Esta aproximación *low-code* y acelerada permitió explorar rápidamente múltiples alternativas y garantizar la eficiencia en la experimentación.

El análisis de los datos reveló una excelente calidad en función de la ausencia total de valores faltantes, y, en paralelo, identificó un desafío significativo: la extrema asimetría positiva y la alta curtosis en las variables de métricas operacionales (como los tiempos de servicio). Estos patrones de distribución, característicos de la ocurrencia de valores atípicos (*outliers*) que representan eventos de servicio prolongado. En la etapa de ingeniería de características, se utilizaron *tsfresh* y *featuretools* para generar masivamente nuevas variables a partir de estadísticas descriptivas y transformaciones primitivas, logrando una posterior reducción de dimensionalidad del 94.3% mediante la combinación de variables con muy baja varianza y el *feature selector* de *autofeat*. A continuación, la fase de *AutoML* (H2O) identificó a un *Stacked Ensemble* de los modelos *gradient boosting machine* (GBM), *distributed random forest* (DRF) y *XGBoost* (XGB) como el modelo con mejor rendimiento predictivo. Por su parte, *PyCaret* determinó un *Huber Regressor* y *AutoKeras* un *Custom Best_AutoKeras_Model*.

Posteriormente, y con el objetivo de ampliar la cantidad de modelos a comparar, se utilizaron tres técnicas de optimización de hiperparámetros de los tres componentes del *ensemble* obtenido con H2O. *Grid search*, optimización bayesiana y *random search* potenciado con muestreo bayesiano fueron las técnicas utilizadas para ello. De estos resultados, se volvieron a conjugar en nuevos *ensembles*. Finalmente, se crearon 12 modelos adicionales a los primeros 3 mencionados. De estos 15 modelos, se seleccionó como el de mayor precisión y potencialidad predictiva, el GBM optimizado con búsqueda aleatoria y muestreo bayesiano.

Para garantizar que el modelo desarrollado pueda operar de manera continua y confiable, se implementó un marco *MLOps*, el cual proporciona la infraestructura necesaria para la reproducibilidad, trazabilidad y sostenibilidad en producción. Esta solución se articuló mediante la orquestación de *pipelines* en *GoCD*, el registro de experimentos y artefactos en *MLflow*, y la gestión de dependencias y versionado de *Python* mediante entornos *Conda* aislados. Esta integración de *MLOps* y *AutoML* asegura que la automatización en la construcción del modelo se traduzca en un activo productivo y auditable. En última instancia, el pronóstico confiable que ofrece este sistema permite a la empresa programar con mayor eficiencia la utilización de camas, ajustar la agenda de cirugías y reducir la proporción de derivaciones a instituciones externas, optimizando recursos y mejorando el servicio a los afiliados.

Referencias

- Akiba, T. et al. (2019). *Optuna: A next-generation hyperparameter optimization framework*. <https://optuna.readthedocs.io/en/stable/index.html>
- Ali, M. (2020). *PyCaret: An open source, low-code machine learning library in Python* (Versión 3.0.4). <https://pycaret.readthedocs.io/en/latest/>
- Andra Ionescu, Z. M. (2024). *autofeat: The automated feature engineering and selection package*. <https://autofeat.readthedocs.io/en/latest/>
- Bais, G. (22 de abril de 2025). *How to Optimize Hyperparameter Search Using Bayesian Optimization and Optuna*. *Neptune.ai*. <https://neptune.ai/blog/how-to-optimize-hyperparameter-search>
- Christ, M. y Blue Yonder GmbH. (2023-2025). *tsfresh: A Python package for feature extraction from time series*. <https://tsfresh.readthedocs.io/en/latest/text/introduction.html#id8>
- Côté, P. O. et al (31 de mayo de 2024). *Data Cleaning and Machine Learning: A Systematic Literature Review*. *arXiv*. <https://doi.org/10.48550/arXiv.2310.01765>
- Dask Development Team (2025). *Dask: Documentation*. <https://docs.dask.org/en/stable/>
- Feature Labs, Inc. (16 de marzo de 2022). *Featuretools Documentation. Release 1.7.0*. https://featuretools.alteryx.com/_downloads/en/v1.7.0/pdf/
- Frazier, P. I. (8 de julio de 2018). *A Tutorial on Bayesian Optimization*. *arXiv*. <https://doi.org/10.48550/arXiv.1807.02811>
- H2O.ai. (2025). *H2O Python Module documentation*. *H2O.ai*. <https://docs.h2o.ai/h2o/latest-stable/h2o-py/docs/index.html>
- Li, L. et al. (18 de junio de 2018). *Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization*. *arXiv*. <https://doi.org/10.48550/arXiv.1603.06560>
- Li, P. et al. (5 de abril de 2021). *CleanML: A study for evaluating the impact of data cleaning on ML classification tasks*. *arXiv*. <https://doi.org/10.48550/arXiv.1904.09483>
- Kreuzberger, D., Kühl, N., y Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, 11, 31866–31879. doi:10.1109/ACCESS.2023.3262138
- Markov, I. L. et al (2023). *Scalable End-to-End ML Platforms: from AutoML to Self-serve*. *arXiv*. <https://doi.org/10.48550/arXiv.2302.14139>
- PyPI (29 de noviembre de 2023). *sweetviz 2.3.1*. *PyPI.org*. <https://pypi.org/project/sweetviz/>

- Seshadri, R. (2020). *AutoViz: The One-Line Automatic Data Visualization Library*. Github.
<https://github.com/AutoViML/AutoViz>
- Symeonidis, G. et al. (2022). *MLOps - Definitions, Tools and Challenges*. arXiv.
<https://doi.org/10.48550/arXiv.2201.00162>
- Watanabe, S. (2025). *Tree-Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance*. arXiv.
<https://doi.org/10.48550/arXiv.2304.11127>
- YData Profiling Development Team (2025). *YData Profiling: Documentation*.
<https://docs.profiling.ydata.ai/latest/>

Anexo 1: CANVAS Business Model



Las prepagas, sanatorios y clínicas enfrentan una gran incertidumbre sobre la cantidad de internaciones no programadas. Esto genera días de saturación, con derivaciones costosas a otros centros, y días de baja ocupación donde los recursos quedan subutilizados. Nuestra propuesta es usar un sistema de predicción basado en datos históricos para anticipar la demanda y mejorar la gestión de camas.

Propuesta de Valor (centro): Lo que ofrecemos es una herramienta de predicción que permite reducir costos y optimizar el porcentaje de ocupación de camas. Conocer con anticipación la demanda de internaciones ayuda a programar cirugías, evitar derivaciones externas y aprovechar mejor los recursos.

Mercado (lado derecho): Los clientes a los que apuntamos son todos los actores del sector: prepagas, sanatorios y clínicas privadas. Vamos a llegar a ellos mediante contacto directo, presentaciones en congresos médicos y marketing digital. La relación que planteamos no es un producto enlatado, sino un acompañamiento personalizado de consultoría e implementación adaptada a cada institución.

Recursos y operaciones (lado izquierdo): Para que esto funcione, el insumo esencial son los datos históricos de internaciones y servicios médicos de cada empresa. Con eso construimos los modelos predictivos. Nuestro trabajo clave está en el desarrollo de los modelos, su implementación y la capacitación de los equipos de la empresa. Contamos además con especialistas médicos y equipos tecnológicos de soporte que son socios clave en la puesta en marcha.

Parte financiera: El modelo de ingresos es claro: se basa en consultoría e implementación, sin costos ocultos ni licencias perpetuas. La estructura de costos está directamente relacionada con las horas de consultoría, el desarrollo del modelo y la capacitación. En definitiva, se trata de un servicio de valor agregado que transforma datos que ya existen en la empresa en una ventaja competitiva real.